

PROYECTO FINAL INTEGRADOR

ESTRUCTURA DE DATOS II

TITULO: SISTEMA DE GESTIÓN ACADÉMICA.

NOMBRE: KATHERINE ALVAREZ AMADOR.

CARRERA: ING. INFORMÁTICA.

Año: 2DO.

FECHA: JUNIO 2026.

Contexto:

En las instituciones de educación superior, los planes de estudio están compuestos por numerosas asignaturas organizadas de manera secuencial. Muchas de estas materias poseen prerequisites, es decir, requieren que el estudiante haya aprobado previamente otras asignaturas antes de poder matricularlas. Debido a la cantidad de materias y relaciones existentes dentro de una carrera universitaria, la gestión y consulta de esta información puede resultar compleja tanto para estudiantes como para coordinadores académicos.

En muchos casos, los estudiantes deben revisar manualmente los planes de estudio para conocer qué materias pueden cursar, cuáles dependen de otras y cuál es la secuencia adecuada para avanzar en la carrera. Este proceso puede generar confusiones, errores de planificación académica y dificultades para comprender las relaciones existentes entre las asignaturas.

Por otra parte, la información académica debe mantenerse organizada para facilitar la búsqueda, actualización y consulta de materias. Cuando el número de asignaturas aumenta, se vuelve necesario utilizar estructuras de datos eficientes que permitan administrar dicha información de manera rápida y ordenada.

Ante esta situación, surge la necesidad de desarrollar un sistema que permita gestionar las materias de un plan de estudios y representar de forma clara las relaciones de dependencia existentes entre ellas, facilitando la consulta y el análisis de la información académica.

Objetivo Específico:

Desarrollar una aplicación en Java que permita gestionar materias académicas mediante el uso integrado de un Árbol Binario de Búsqueda (ABB) y un Grafo Dirigido, proporcionando funcionalidades para registrar, buscar, modificar y eliminar materias, así como representar y analizar las relaciones de prerequisites existentes entre ellas.

El sistema deberá permitir almacenar la información de manera organizada, facilitar la consulta de materias específicas y mostrar las dependencias académicas mediante algoritmos de recorrido de grafos, contribuyendo a una mejor comprensión de la estructura curricular de una carrera universitaria.

Justificación del Uso de Árboles y Grafos:

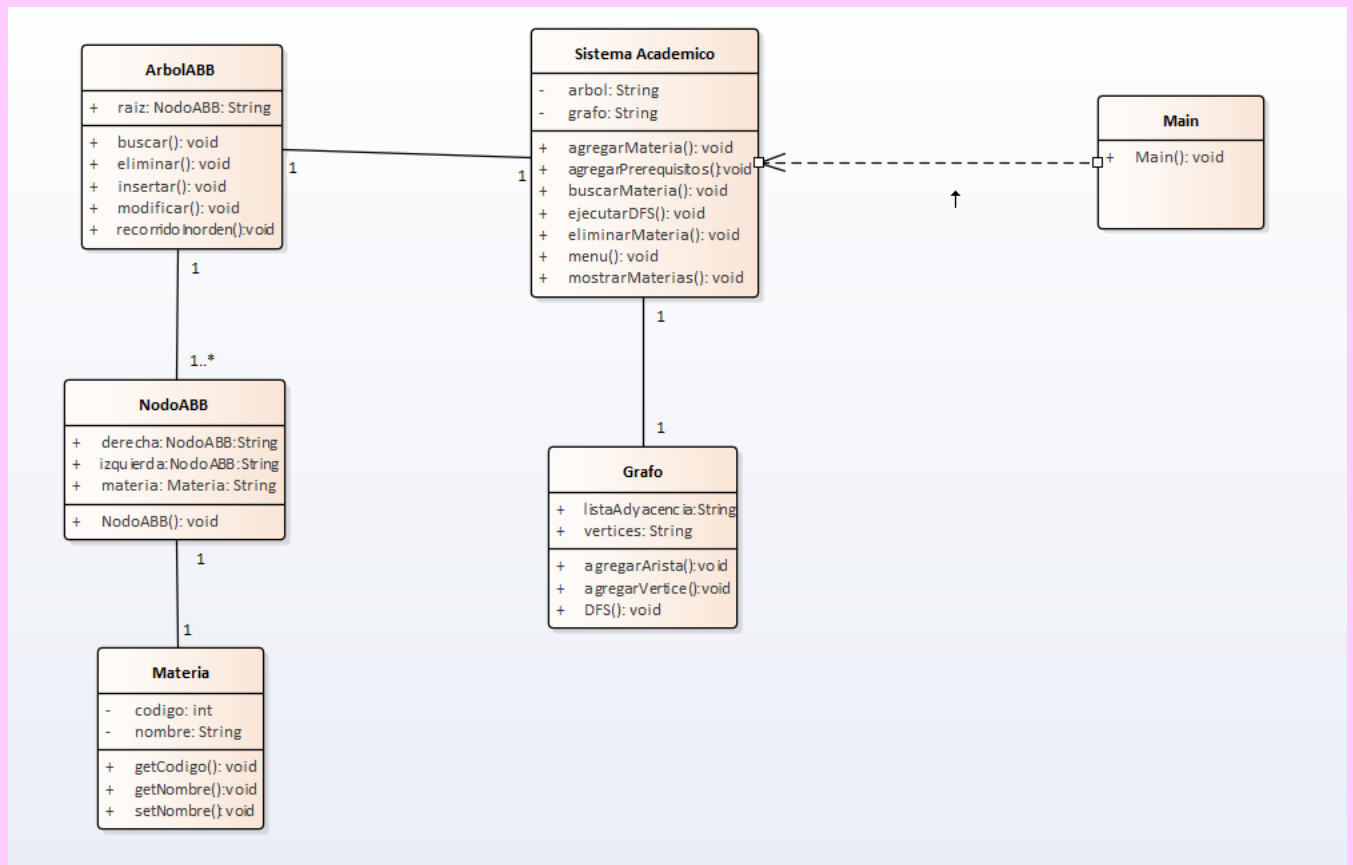
La elección de las estructuras de datos utilizadas en este proyecto responde directamente a las necesidades de la problemática planteada.

El Árbol Binario de Búsqueda (ABB) se utilizará para almacenar las materias académicas de forma ordenada según su código identificador. Esta estructura permite realizar operaciones de inserción, búsqueda, modificación y eliminación de manera eficiente, facilitando la administración de la información académica. Además, mediante sus recorridos es posible visualizar las materias organizadas de forma ordenada, lo cual resulta útil para la gestión del sistema.

Por otra parte, las relaciones de prerrequisitos entre materias representan una estructura de conexión que puede modelarse de manera natural mediante un Grafo Dirigido. En este caso, cada vértice del grafo representará una materia y cada arista indicará una relación de dependencia entre dos asignaturas. Gracias a esta representación será posible analizar las conexiones existentes y recorrer las dependencias utilizando algoritmos propios de los grafos, como DFS.

La integración de ambas estructuras constituye uno de los aspectos fundamentales del proyecto. Mientras el ABB se encargará de gestionar y organizar la información de las materias, el grafo permitirá representar y analizar las relaciones académicas entre ellas. De esta forma, ambas estructuras colaboran para resolver una misma problemática, cumpliendo con los objetivos planteados y demostrando la aplicación práctica de los conocimientos adquiridos en la asignatura Estructura de Datos II.

Diagrama de clases:



Decisiones de Diseño:

Para resolver la problemática de gestión académica se decidió utilizar dos estructuras de datos principales:

Árbol Binario de Búsqueda (ABB):

El ABB se empleó para almacenar las materias registradas en el sistema.

Las materias se organizan mediante su código, permitiendo:

- Inserción eficiente de materias.
- Búsqueda rápida por código.
- Modificación de información.
- Eliminación de materias.
- Visualización ordenada mediante recorrido InOrden.

La utilización del ABB mejora la organización y el acceso a la información académica.

Grafo:

El grafo se utilizó para modelar los prerrequisitos entre materias.

Cada vértice representa una materia y cada arista representa una relación de prerrequisito.

Esto permite:

- Registrar dependencias académicas.
- Consultar prerrequisitos.
- Visualizar relaciones entre materias.
- Realizar recorridos DFS para explorar dependencias.

Relación entre las Estructuras:

El sistema integra un Árbol Binario de Búsqueda y un Grafo para representar diferentes aspectos de la información académica.

Función del ABB:

El ABB administra las materias registradas mediante su código único, garantizando búsquedas y actualizaciones eficientes.

Función del Grafo:

El Grafo administra las relaciones de dependencia entre materias, representando los prerrequisitos necesarios para cursar determinadas asignaturas.

Integración:

Cuando una materia es registrada:

- o Se almacena en el ABB.
- o Se agrega como vértice en el Grafo.

Cuando se registra un prerrequisito:

- o Se verifica que ambas materias existan.
- o Se crea una arista entre ellas en el Grafo.

De esta forma, ambas estructuras trabajan conjuntamente para gestionar tanto la información de las materias como sus relaciones académicas.

Implementación:

En esta sección se presentan los fragmentos de código más importantes del sistema desarrollado. Se muestran las implementaciones principales del Árbol Binario de Búsqueda (ABB) utilizado para gestionar las materias y del Grafo utilizado para representar los prerrequisitos. Además, se describen los algoritmos empleados para la inserción, búsqueda y recorrido de las estructuras de datos, así como algunas validaciones implementadas para garantizar la integridad de la información.

```
// INSERTAR

public void insertar(Materia materia) { raiz = insertarRec(raiz, materia); }

private arbol.NodoABB insertarRec(arbol.NodoABB actual, Materia materia) { 3 usages

    if (actual == null) {
        return new arbol.NodoABB(materia);
    }

    if (materia.getCodigo() < actual.materia.getCodigo()) {

        actual.izquierda =
            insertarRec(actual.izquierda, materia);

    } else if (materia.getCodigo() > actual.materia.getCodigo()) {

        actual.derecha =
            insertarRec(actual.derecha, materia);

    }

    return actual;
}
```

Este método permite insertar una materia en el Árbol Binario de Búsqueda (ABB). Para ello, compara el código de la nueva materia con los códigos ya almacenados en el árbol. Si el código es menor, se inserta en la rama izquierda; si es mayor, se inserta en la rama derecha. Cuando encuentra una posición vacía, crea un nuevo nodo con la materia. De esta forma, las materias quedan organizadas de manera ordenada según su código.

```
// BUSCAR

public Materia buscar(int codigo) { 5 usages

    arbol.NodoABB resultado =
        buscarRec(raiz, codigo);

    if (resultado != null) {
        return resultado.materia;
    }

    return null;
}
```

```
private arbol.NodoABB buscarRec(arbol.NodoABB actual, int codigo) {

    if (actual == null ||
        actual.materia.getCodigo() == codigo) {

        return actual;
    }

    if (codigo < actual.materia.getCodigo()) {

        return buscarRec(
            actual.izquierda,
            codigo
        );
    }

    return buscarRec(
        actual.derecha,
        codigo
    );
}
```

Este método permite buscar una materia dentro del Árbol Binario de Búsqueda (ABB) utilizando su código. La búsqueda se realiza de forma recursiva: si el código buscado es igual al del nodo actual, se devuelve la materia; si es menor, se continúa la búsqueda por la rama izquierda; y si es mayor, por la rama derecha. Si se llega a un nodo nulo, significa que la materia no existe en el árbol. Gracias a la organización del ABB, la búsqueda es más eficiente que recorrer todos los elementos.

```
//AGREGAR ARISTA

public boolean agregarArista(String origen, String destino) {

    int i = vertices.indexOf(origen);
    int j = vertices.indexOf(destino);

    if(i != -1 && j != -1) {

        adyacencias.get(i).add(j);

        return true;
    }

    return false;
}
```

Este método permite agregar una relación de prerequisite entre dos materias dentro del grafo. Primero busca la posición de la materia de origen y de la materia de destino en la lista de vértices. Si ambas existen, se agrega una arista desde la materia de origen hacia la materia de destino y el método devuelve true. En caso contrario, devuelve false, indicando que una o ambas materias no están registradas en el sistema. De esta manera, se representan las dependencias académicas entre las materias.


```
//RECORRIDO DFS

public void DFS(String inicio) { 1 usage

    boolean[] visitado = new boolean[vertices.size()];

    int indice = vertices.indexOf(inicio);

    if (indice != -1) {
        dfsRec(indice, visitado);
    }
}

private void dfsRec(int vertice, boolean[] visitado) { 2 usages

    visitado[vertice] = true;

    System.out.println(vertices.get(vertice));

    for (Integer vecino : adyacencias.get(vertice)) {

        if (!visitado[vecino]) {
            dfsRec(vecino, visitado);
        }
    }
}
```

Este método se encarga de recorrer las materias almacenadas en el grafo utilizando el algoritmo DFS. El recorrido comienza desde la materia que el usuario indique y va visitando las materias relacionadas una por una hasta recorrer todas las posibles. Para evitar que una misma materia se visite varias veces, se utiliza un arreglo llamado visitado. Además, el método dfsRec() realiza el recorrido de forma recursiva, mostrando cada materia encontrada durante el proceso. Esto permite visualizar las relaciones y dependencias entre las materias registradas en el sistema.

```
if (codigo < 100 || codigo > 999) {  
  
    System.out.println(  
        "Error: el codigo debe tener exactamente 3 cifras."  
    );  
  
    continue;  
}
```

. En este fragmento se verifica que el código ingresado para la materia tenga tres cifras. Si el usuario introduce un número menor que 100 o mayor que 999, el sistema muestra un mensaje de error y no permite continuar hasta que se ingrese un código válido. Esta validación se realizó para evitar errores y asegurar que todas las materias tengan un formato de código consistente dentro del sistema.

Pruebas y Resultados:

Con el objetivo de verificar el correcto funcionamiento del sistema desarrollado, se realizaron diferentes casos de prueba para comprobar las operaciones principales implementadas en el Árbol Binario de Búsqueda (ABB) y en el Grafo. A continuación, se presentan los casos de prueba más representativos.

Caso de Prueba 1: Registro de una Materia

Objetivo: Verificar que el sistema permita agregar una materia correctamente.

Entrada:

- Opción: Agregar materia
- Código: 101
- Nombre: Cálculo III

Salida esperada:

Materia agregada correctamente.

Resultado obtenido:

La materia fue registrada exitosamente en el sistema y almacenada tanto en el Árbol Binario de Búsqueda como en el Grafo.

Captura de ejecución: ´

```
===== SISTEMA ACADEMICO =====
1. Agregar materia
2. Buscar materia
3. Modificar materia
4. Eliminar materia
5. Mostrar materias
6. Agregar prerrequisito
7. Consultar prerrequisitos
8. Ejecutar DFS
9. Salir
1
Codigo (3 cifras): 100
Nombre: Calculo III
Materia agregada correctamente.
```

Caso de Prueba 2: Registro de Prerrequisitos

Objetivo: Verificar que el sistema permita establecer relaciones entre materias.

Entrada:

- Materia origen: Cálculo III

- Materia destino: Cálculo IV

Salida esperada:

Prerrequisito agregado correctamente.

Resultado obtenido:

La relación entre ambas materias fue almacenada correctamente en el Grafo, permitiendo representar la dependencia académica entre ellas.

Captura de ejecución:

```
===== SISTEMA ACADEMICO =====
1. Agregar materia
2. Buscar materia
3. Modificar materia
4. Eliminar materia
5. Mostrar materias
6. Agregar prerrequisito
7. Consultar prerrequisitos
8. Ejecutar DFS
9. Salir
6
Materia origen: Calculo III
Materia destino: Calculo IV
Prerequisito agregado correctamente.
```

Caso de Prueba 3: Recorrido DFS

Objetivo: Verificar el funcionamiento del algoritmo DFS para recorrer las relaciones entre materias.

Entrada:

- Materia inicial: Cálculo III

Salida esperada:

Se muestran las materias relacionadas con la materia inicial siguiendo el recorrido en profundidad.

Resultado obtenido:

El algoritmo DFS recorrió correctamente las materias conectadas, demostrando el funcionamiento adecuado del Grafo y de las relaciones de prerequisites registradas.

Captura de ejecución:

```
===== SISTEMA ACADEMICO =====
1. Agregar materia
2. Buscar materia
3. Modificar materia
4. Eliminar materia
5. Mostrar materias
6. Agregar prerequisite
7. Consultar prerequisites
8. Ejecutar DFS
9. Salir
8
Materia inicial: Calculo III
Calculo III
Calculo IV
```

Análisis de Resultados:

Los casos de prueba realizados permitieron comprobar el correcto funcionamiento de las principales operaciones del sistema. Se verificó el registro de materias mediante el ABB, la creación de relaciones de prerequisites mediante el Grafo y el recorrido de dichas relaciones utilizando el algoritmo DFS. Los resultados obtenidos coincidieron con los resultados esperados, demostrando que la solución implementada cumple con los objetivos planteados para la gestión académica.

CONCLUSIONES:

Con la realización de este proyecto pude comprender mejor el funcionamiento de los Árboles Binarios de Búsqueda y los Grafos, aplicándolos en un caso práctico de gestión académica. Además, aprendí cómo utilizar el algoritmo DFS para recorrer las relaciones entre las materias.

Una de las principales dificultades fue lograr que las diferentes partes del sistema trabajaran correctamente entre sí, especialmente la relación entre las materias y sus prerequisites, así como la implementación de las validaciones necesarias.

Como mejora futura, se podría agregar una interfaz gráfica y ampliar las funcionalidades del sistema para gestionar más información académica.

En general, este proyecto me permitió reforzar mis conocimientos sobre estructuras de datos y programación orientada a objetos.

REFERENCIAS BIBLIOGRAFICAS:

- ✓ Conferencia: Árbol de Búsqueda Binaria (ABB).
- ✓ Conferencia: Grafos
- ✓ IA: Chat GPT.
- ✓ Vídeos youtube.